

MD005 - Sistemes Basats en el Coneixement

Quarta Entrega: Optimització



Alumne: Gerard Pascual Fontanilles

Professora: Jordina Torrents

Data d'entrega: 6 de febrer de 2026

Informació sobre el conjunt de dades

Per fer aquest treball he tornat a utilitzar el conjunt de dades de pokemons. Extret de:
<https://www.kaggle.com/datasets/hamzacyberpatcher/data-of-1010-pokemons>

Tornem a repassar el conjunt de dades: conté informació sobre 1025 pokemons diferents, incloent-hi les seves estadístiques bàsiques, tipus, generació i si evolucionen d'algun altre pokemon.

Les característiques principals del conjunt de dades són:

- Pokemon's id
- Pokemon's name
- Pokemon's rank (whether it is legendary, mythical, baby or an ordinary pokemon)
- Pokemon's generation
- The pokemon from which this pokemon evolved from
- It's primary and secondary type as type1 and type2
- It's base stats
- It's height (decimetres) and weight (hectograms)
- It's abilities (Note: if a pokemon has multiple abilities such as overgrow and chlorophyll than they are separated by a space and if there is a space in the name of the ability it has been replaced by a hyphen for example a pokemon with abilities blaze and solar power it is given as blaze solar-power)
- A little description of the pokemon in english (description of pokemons from 1009 is marked as "Not Available").

Imports necessaris

```
In [1]: # =====  
# Standard Library  
# =====  
import os  
import time  
import warnings  
  
# =====
```

```

# Core Data & Visualization
# =====
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# =====
# Scikit-learn - Model Selection & Evaluation
# =====
from sklearn.model_selection import (
    train_test_split,
    StratifiedKFold,
    cross_val_score,
)

from sklearn.metrics import (
    classification_report,
    f1_score,
    accuracy_score,
    ConfusionMatrixDisplay,
    RocCurveDisplay
)

# =====
# Scikit-learn - Preprocessing & Pipelines
# =====
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.base import clone

# =====
# Scikit-learn - Models
# =====
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, VotingClassifier

# =====
# Additional ML Libraries
# =====
from xgboost import XGBClassifier
from tpot import TPOTClassifier
import optuna

# =====
# Inspection
# =====
from sklearn.inspection import permutation_importance

```

```

/home/sirp/Documents/MUDS/SBC/.venv/lib/python3.14/site-packages/stopit/__init__.py:1
0: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/
en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early
as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
import pkg_resources

```

1. Preparació del Conjunt de Dades: Estratègia de Fusió

Per a aquesta pràctica, centrada en l'exploració exhaustiva d'algoritmes d'optimització d'hiperparàmetres, hem decidit utilitzar el problema de **Classificació Binària** (Llegendari vs. No

Llegendari). Aquest problema presenta un desequilibri de classes (9% positius) que fa que l'ajust fi dels paràmetres sigui crític per maximitzar mètriques com l'F1-Score.

En lloc de repetir exactament el preprocessament de l'entrega de classificació original, hem dissenyat una **estratègia híbrida**. Hem fusionat les millors tècniques de neteja de la 2a entrega amb les transformacions estadístiques avançades de la 3a entrega, afegint-hi a més variables d'enginyeria que anteriorment només havíem testejat experimentalment.

L'objectiu és oferir als algorismes d'optimització un espai de característiques (**Feature Space**) més ric i complex, cosa que permet que tècniques de selecció de característiques (com els algorismes genètics) tinguin més material per treballar.

1.1. Definició de la Variable Objectiu (Target)

Mantenim l'enfocament binari original:

- **Target (`is_legendary`)**: 1 si el rang és 'Legendary' o 'Mythical', 0 si és 'Ordinary' o 'Baby'.

1.2. Construcció de la Matriu de Característiques (X)

Hem processat les dades aplicant les següents millores respecte a l'entrega de classificació original:

- **Transformació Logarítmica (Herència de Regressió)**: A l'entrega de regressió vam demostrar que les variables `weight` i `height` tenien un biaix positiu extrem. Per corregir la distribució i facilitar la convergència d'algorismes sensibles a la normalitat (com SVM o Regressió Logística), substituïm les originals per la seva transformació logarítmica: `log_weight` i `log_height`.
- **Recuperació de `type2` (Herència de Regressió)**: A diferència de la primera entrega on vam eliminar `type2`, aquí la mantenim imputant els valors nuls com a categoria `'None'`. Això permetrà als algorismes de selecció decidir si el segon tipus aporta informació rellevant per detectar llegendaris.
- **Enginyeria de Variables (Recuperació Experimental)**: Incorporem variables sintètiques que anteriorment havíem reservat per a experiments secundaris. La hipòtesi és que els llegendaris poden tenir ràtios de poder distintius:
 - `atk_ratio` : Relació Atac Físic / Especial.
 - `def_ratio` : Relació Defensa Física / Especial.
 - `bmi` : Índex de massa corporal (densitat del Pokémon).
 - `bulk` : Resistència total (`hp` + `def` + `spdef`).
- **Evolució i Neteja Standard**: Mantenim la variable `is_base` (binària) per indicar si el Pokémon evoluciona d'un altre, ja que va demostrar ser un predictor molt fort.
- **Eliminem variables redundants o sorolloses**:
 - Identificadors: `id`, `name`, `desc`.
 - Redundàncies lineals: `total` (suma de les altres stats) i les originals `height` / `weight` (substituïdes per la versió logarítmica).

1.3. Resum de l'Espai de Cerca

El dataset resultant per a l'optimització conté les següents variables predictores:

1. **Físiques**: `hp`, `atk`, `def`, `spatk`, `spdef`, `speed`.

2. **Transformades:** `log_weight` , `log_height` .
3. **Enginyeria:** `bmi` , `bulk` , `atk_ratio` , `def_ratio` .
4. **Categòriques (One-Hot):** `generation` , `type1` , `type2` , `is_base` .

Aquest conjunt enriqueix és el punt de partida ideal per comparar l'eficàcia de mètodes com **Tree-structured Parzen Estimator (TPE)** o **Population-based Search**, ja que hauran de gestionar un major nombre de dimensions i possibles interaccions no lineals.

```
In [2]: df = pd.read_csv('pokemons.csv')
```

```
In [3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    1025 non-null   int64
1   name                  1025 non-null   object
2   rank                  1025 non-null   object
3   generation            1025 non-null   object
4   evolves_from          1025 non-null   object
5   type1                 1025 non-null   object
6   type2                 526 non-null    object
7   hp                    1025 non-null   int64
8   atk                   1025 non-null   int64
9   def                   1025 non-null   int64
10  spatk                 1025 non-null   int64
11  spdef                 1025 non-null   int64
12  speed                 1025 non-null   int64
13  total                 1025 non-null   int64
14  height                1025 non-null   int64
15  weight                1025 non-null   int64
16  abilities             1025 non-null   object
17  desc                  1025 non-null   object
dtypes: int64(10), object(8)
memory usage: 144.3+ KB
```

```
In [4]: df['type2'] = df['type2'].fillna('None')
```

```
In [5]: df['is_base'] = df['evolves_from'].apply(lambda x: 1 if x == 'nothing' else 0)
```

```
In [6]: df['log_height'] = np.log1p(df['height'])
df['log_weight'] = np.log1p(df['weight'])
```

```
In [7]: df['atk_ratio'] = df['atk'] / (df['spatk'] + 1)
df['def_ratio'] = df['def'] / (df['spdef'] + 1)
df['bmi'] = df['weight'] / (df['height'] + 1)
df['bulk'] = df['hp'] + df['def'] + df['spdef']
```

```
In [8]: mapa_rank = {
    'legendary': 1,
    'mythical': 1,
    'ordinary': 0,
    'baby': 0
}
df['is_legendary'] = df['rank'].map(mapa_rank).fillna(0).astype(int)
```

```
In [9]: cols_to_drop = [
        'id', 'name', 'desc', 'abilities', 'evolves_from',
        'total', 'rank', 'height', 'weight'
    ]
```

```
In [10]: existing_drop = [c for c in cols_to_drop if c in df.columns]
df.drop(columns=existing_drop, inplace=True)
```

```
In [11]: print(f"Columnes eliminades: {existing_drop}")
print("Noves variables creades: log_height, log_weight, atk_ratio, def_ratio, bmi, bu")
df.head()
```

```
Out[11]:
```

	generation	type1	type2	hp	atk	def	spatk	spdef	speed	is_base	log_height	log_weight	a
0	generation-	grass	poison	45	49	49	65	65	45	1	2.079442	4.248495	0
1	generation-	grass	poison	60	62	63	80	80	60	0	2.397895	4.875197	0
2	generation-	grass	poison	80	82	83	100	100	80	0	3.044522	6.908755	0
3	generation-	fire	None	39	52	43	60	50	65	1	1.945910	4.454347	0
4	generation-	fire	None	58	64	58	80	65	80	0	2.484907	5.252273	0

```
In [12]: numeric_df = df.select_dtypes(include=[np.number])

corr_matrix = numeric_df.corr()

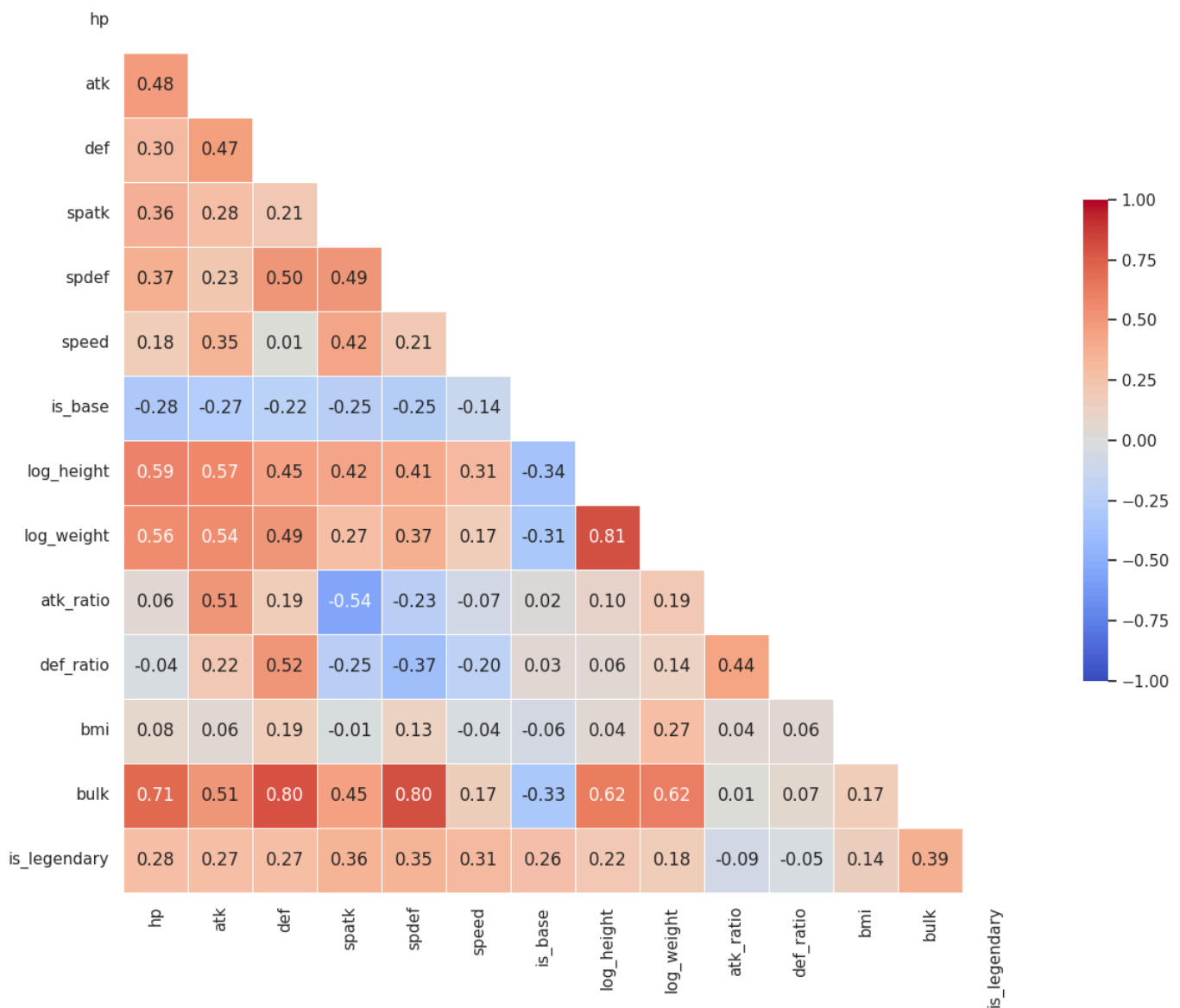
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

plt.figure(figsize=(14, 12))
sns.set_theme(style="white")

heatmap = sns.heatmap(
    corr_matrix,
    mask=mask,
    annot=True,
    fmt=".2f",
    cmap='coolwarm',
    vmax=1, vmin=-1,
    center=0,
    square=True,
    linewidths=.5,
    cbar_kws={"shrink": .5}
)

plt.title('Mapa de Correlació: Variables Originals vs Enginyeria', fontsize=16)
plt.show()
```

Mapa de Correlació: Variables Originals vs Enginyeria



Interpretació del Mapa de Calor

1. L'Èxit de la Variable **bulk** (Resistència Total):

- La variable sintètica **bulk** (**hp** + **def** + **spdef**) mostra una correlació de **0.39** amb el target **is_legendary** .
- Aquesta és la **correlació individual més alta** de tot el dataset, superant a variables clàssiques com l'Atac Especial (**spatk** : 0.36) o la Defensa Especial (**spdef** : 0.35).
 - Això confirma que la característica més definitiva d'un Pokémon llegendari no és tenir una sola estadística alta, sinó tenir una resistència general massiva. Haver creat aquesta variable ajudarà enormement als models a detectar aquest patró.

2. **Relació Pes-Alçada vs. Target:** Tot i que **log_weight** i **log_height** tenen una forta correlació entre elles (**0.81**), el seu comportament respecte a l'objectiu és diferent. El pes té una correlació superior (0.29) a l'alçada (0.24) amb ser llegendari. Això suggereix que la "massivitat" és un predictor més fiable que la simple grandària, aportant matisos diferents.

3. **Irrellevància dels Ràtios:** Variables com **atk_ratio** (-0.09) o **bmi** (0.14) mostren correlacions molt febles. Això indica que els llegendaris no es defineixen per la seva "especialització" (ser molt físics o molt especials) ni per la seva densitat, sinó per la magnitud bruta de les seves estadístiques.

Conservació de Variables

Hem detectat parells de variables amb **alta multicolinealitat**, concretament:

- `bulk` vs. `def` / `spdef` (Correlació 0.80).
- `log_weight` vs. `log_height` (Correlació 0.81).

Malgrat aquests valors alts, la nostra decisió tècnica és **MANTENIR totes les variables** i no eliminar-ne cap manualment.

Justificació de la Decisió:

1. **Informació Única Residual:** Una correlació de 0.80 implica que encara hi ha un 20-30% de variància no compartida. Per exemple, un Pokémon com *Blissey* té un `bulk` altíssim però una `def` molt baixa. Si eliminéssim `def` per quedar-nos només amb `bulk`, el model perdria la capacitat de distingir entre un tanc de vida i un tanc d'armadura.
2. **Robustesa dels Models Que Triarem:**
 - **SVM:** Utilitza regularització L2 (paràmetre `C`), que penalitza automàticament els pesos de les variables redundants, evitant que la multicolinealitat desestabilitzi el model.
 - **Random Forest / TPOT:** Els models basats en arbres són immunes a la multicolinealitat. Si dues variables són similars, l'arbre triarà la millor (`bulk`) per al tall principal i utilitzarà l'altra (`def`) només en branques secundàries on aportí informació extra.
3. **Optimització Automàtica:** Atès que utilitzarem **Algoritmes Genètics (TPOT)**, és més eficient proporcionar totes les opcions (variables originals + derivades) i deixar que l'algorisme realitzi la selecció de característiques (*Feature Selection*) de manera matemàtica durant l'evolució, en lloc d'aplicar un criteri manual subjectiu.

2. Establiment de la Línia Base (*Baseline*)

Abans d'aplicar estratègies complexes d'optimització d'hiperparàmetres, és imprescindible establir un punt de partida o **Línia Base**. Això ens permetrà quantificar objectivament quant millora el rendiment gràcies a l'optimització respecte a una implementació estàndard.

En aquesta secció, rescatem els quatre algorismes principals utilitzats en l'entrega de Classificació per executar-los en la seva configuració "**Vanilla**" (paràmetres per defecte de *scikit-learn*):

1. **Regressió Logística:** Com a model lineal de referència.
2. **Support Vector Machine (SVM):** Per la seva capacitat de generar hiperplans complexos.
3. **K-Nearest Neighbors (KNN):** Basat en distància i similitud.
4. **Random Forest:** Com a mètode d'ensamblatge.

Implementació Tècnica amb `Pipeline`

Tot i que utilitzarem la configuració per defecte dels hiperparàmetres, per garantir que els algorismes funcionin tècnicament i no donin errors d'execució, és necessari gestionar la heterogeneïtat de les dades (variables numèriques transformades, ràtios petites i categories textuais).

Per fer-ho, implementem un `ColumnTransformer` integrat en un `Pipeline` que automatitza:

- **Estandarització (`StandardScaler`):** Crítica per a SVM i KNN, ja que les variables tenen magnituds molt diferents (ex: `bmi` vs `hp`).
- **Codificació (`OneHotEncoder`):** Necessària per traduir `type1`, `type2` i `generation` a format numèric.

L'única excepció a la configuració "per defecte" serà l'activació del paràmetre

`class_weight='balanced'` (en els models que ho permeten). Com vam demostrar en l'entrega anterior, sense aquesta correcció els models tendeixen a ignorar la classe minoritària (Llegendaris, ~9%), la qual cosa invalidaria la comparació de mètriques com l'F1-Score.

```
In [13]: target = 'is_legendary'
features = [col for col in df.columns if col != target]
```

```
In [14]: X = df[features]
y = df[target]
```

```
In [15]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)
```

```
In [16]: cols_num = ['hp', 'atk', 'def', 'spatk', 'spdef', 'speed',
    'log_height', 'log_weight', 'bmi', 'bulk', 'atk_ratio', 'def_ratio']
cols_cat = ['generation', 'type1', 'type2']
cols_bin = ['is_base']
```

```
In [17]: preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), cols_num),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cols_cat),
        ('bin', 'passthrough', cols_bin)
    ]
)
```

```
In [18]: results_baseline = []

models_general = {
    "Logistic Regression": LogisticRegression(max_iter=1000, class_weight='balanced',
    "SVM": SVC(class_weight='balanced', probability=True, random_state=42),
    "Random Forest": RandomForestClassifier(class_weight='balanced', random_state=42)
}

for name, model in models_general.items():
    try:
        clf = Pipeline(steps=[
            ('preprocessor', preprocessor),
            ('classifier', model)
        ])

        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)

        results_baseline.append({
            'Model': name,
            'Accuracy': accuracy_score(y_test, y_pred),
            'F1_Macro': f1_score(y_test, y_pred, average='macro'),
            'F1_Target': f1_score(y_test, y_pred, pos_label=1)
        })

    except Exception as e:
        print(f"Error en {name}: {e}")

try:
    X_train_dense = preprocessor.fit_transform(X_train).toarray()
    X_test_dense = preprocessor.transform(X_test).toarray()

    knn = KNeighborsClassifier(n_neighbors=5, n_jobs=-1)
    knn.fit(X_train_dense, y_train)
```



```

y_pred_knn = knn.predict(X_test_dense)

results_baseline.append({
    'Model': 'KNN (Dense)',
    'Accuracy': accuracy_score(y_test, y_pred_knn),
    'F1_Macro': f1_score(y_test, y_pred_knn, average='macro'),
    'F1_Target': f1_score(y_test, y_pred_knn, pos_label=1)
})

except Exception as e:
    print(f"Error en KNN: {e}")

print(f"{'MODEL':<25} | {'ACCURACY':<10} | {'F1-SCORE (Macro)':<15} | {'F1-SCORE (Tar
print("-" * 80)

for res in results_baseline:
    print(f"{'Model':<25} | {'Accuracy':<10} | {'F1-SCORE (Macro)':<15} | {'F1-SCORE (Target 1)':<15}")

```

MODEL	ACCURACY	F1-SCORE (Macro)	F1-SCORE (Target 1)
Logistic Regression	0.9318	0.8328	0.7042
SVM	0.9416	0.8338	0.7000
Random Forest	0.9545	0.8287	0.6818
KNN (Dense)	0.9416	0.7700	0.5714

```
In [19]: df_results_baseline = pd.DataFrame(results_baseline)
```

2.1. Anàlisi dels Resultats de la Línia Base

L'execució dels models amb la seva configuració per defecte (*Vanilla*) ens ofereix conclusions molt valuoses sobre la qualitat del nostre nou dataset fusionat. Els resultats (amb un **F1-Score al voltant de 0.70** per als millors models) són significativament superiors als de la primera entrega de classificació, validant l'estratègia de preprocessament.

Destaquem tres punts clau d'aquesta taula de resultats:

1. La Resurrecció del KNN (Validació de la Transformació Logarítmica)

El canvi més notable respecte a experiències anteriors és el rendiment del **K-Nearest Neighbors**. En l'entrega original, el KNN tenia un rendiment molt pobre ($F1 \sim 0.34$) a causa dels outliers en pes i alçada que distorsionaven les distàncies euclidianes. En aquesta prova, amb un **F1-Score de 0.57**, demostrem que l'aplicació de `np.log1p` combinada amb l'`StandardScaler` ha corregit la geometria de l'espai de dades, fent que els "veïns" siguin realment representatius. Tot i això, continua sent el model amb pitjor rendiment i el més ineficient computacionalment, motiu pel qual no passarà a la fase d'optimització avançada.

2. La Paradoxa de l'Accuracy

Observem que el **Random Forest** obté la millor *Accuracy* (**95.45%**) però es veu superat per la Regressió Logística en l'*F1-Score* de la classe objectiu (0.68 vs 0.70). Això confirma el risc del desequilibri de classes: el Random Forest està optimitzant l'encert global (predint molt bé els Pokémon ordinaris) però és més conservador a l'hora d'arriscar-se a classificar un Llegendari. Això reforça la nostra decisió d'utilitzar l'**F1-Score (Target 1)** com a única mètrica vàlida per a la fase d'optimització.

3. Domini dels Models Lineals/Geomètrics

Tant la **Regressió Logística (0.7042)** com l'**SVM (0.7000)** lideren el rànquing base. Això suggereix que la frontera entre un Pokémon "Llegendari" i un "Ordinari", un cop aplicades les variables d'enginyeria (`atk_ratio`, `bulk`, etc.), és raonablement lineal o definible per hiperplans.

3. Selecció d'Estratègies d'Optimització

L'objectiu d'aquesta fase és superar la nostra *Línia Base* ($F1\text{-Score} \approx 0.70$) cercant la configuració òptima d'hiperparàmetres. Per fer-ho, hem analitzat diferents algorismes disponibles i hem realitzat una selecció estratègica basada en l'eficiència i la naturalesa del nostre conjunt de dades.

3.1. Algorismes Descartats i Justificació

Abans de definir els mètodes escollits, és important justificar per què hem descartat certes opcions clàssiques o com hem integrat altres conceptes:

1. Grid Search (Cerca en Reixa): Descartat.

- És un mètode de "força bruta" que prova totes les combinacions possibles. Amb paràmetres continus (com `C` o `gamma` en un SVM), el Grid Search és extremadament ineficient i lent, ja que creix exponencialment amb cada nou paràmetre.

2. Random Search (Cerca Aleatòria): Descartat.

- Tot i ser millor que el Grid, funciona com una "loteria": no aprèn de les iteracions anteriors. Si troba una configuració bona, no explora el seu veïnatge, sinó que salta a un punt aleatori, perdent oportunitats de convergència fina.

3.2. Mètodes Seleccionats i Estratègia Experimental

Per superar la barrera del **0.70** d'F1-Score establerta pel *Baseline*, no ens limitarem a ajustar un únic model. Hem dissenyat una arquitectura experimental que ataca el problema de classificació des de **tres paradigmes matemàtics completament diferents**.

L'objectiu no és només trobar el millor model, sinó comparar quina aproximació (Geometria, Força Bruta o Evolució) resol millor el problema dels Pokémon Llegendaris.

Mètode 1: SVM + Optuna

Com que l'SVM ha estat el guanyador tècnic del baseline, és el candidat ideal per a l'ajust fi (*Fine Tuning*). L'SVM depèn críticament de trobar la forma exacta de l'hiperplà que separa les dades en dimensions superiors.

Per fer-ho, utilitzem l'**Optimització Bayesiana** mitjançant un **Tree-structured Parzen Estimator (TPE)**. A diferència de la cerca aleatòria (que prova sort), el TPE construeix un model de probabilitat per predir quina combinació d'hiperparàmetres té més opcions de millorar l'F1-Score basant-se en l'aprenentatge de les iteracions anteriors.

- **Espai de Cerca (SVM):**
 - `C` (Regularització): Distribució logarítmica (α). Controla la penalització dels errors.
 - `gamma` (Coeficient del Kernel): Distribució logarítmica (α). Controla l'abast de la influència de cada punt.
 - `kernel`: Categòric `['rbf', 'poly']`. (Radial vs Polinòmic).

Mètode 2: XGBoost - GPU

Abandonem la geometria i passem als Arbres de Decisió. Tot i que el Random Forest ha fallat al baseline, implementem l'estat de l'art: **Extreme Gradient Boosting**. Aquest mètode construeix arbres de manera seqüencial, on cada nou arbre intenta corregir els errors residuals dels anteriors.

- L'ús de la **GPU (NVIDIA CUDA)** ens permet entrenar models massius en segons. A més, XGBoost gestiona internament els valors nuls i, clau per a nosaltres, permet assignar pesos a les classes.
- **Configuració del Model:**
 - `n_estimators`: 500 (Un nombre elevat gràcies a l'acceleració GPU).
 - `scale_pos_weight`: Paràmetre crític per penalitzar més els errors en la classe minoritària (Llegendaris), corregint el defecte del Random Forest.
 - `tree_method`: `'hist'` (Mètode d'histogrames per a màxima velocitat).

Mètode 3: AutoML - TPOT

Inspirat en la biologia evolutiva. En aquest experiment, no assumim quin és el millor model. El TPOT genera una població inicial de *pipelines* aleatoris (que poden incloure PCA, Selecció de Features, Random Forest, SVM, etc.). Els millors "sobreviuen" i es reproduïxen.

- Hem introduït moltes variables d'enginyeria noves (`atk_ratio`, `bulk`, `log_weight`). Els algorismes genètics realitzen una **Selecció de Característiques** automàtica, decidint quines variables aporten valor i quines són soroll.
- **Configuració Evolutiva:**
 - `Generations`: 5 (Iteracions del cicle de vida).
 - `Population Size`: 20 individus (Models competint simultàniament).

- **Mutation/Crossover** : Operadors genètics estàndard per equilibrar exploració i explotació.

Mètode 4: Unificació (Voting Classifier)

Finalment, implementem una estratègia de "Saviesa de la Multitud" (*Wisdom of Crowds*). Creem un meta-model que combina les prediccions dels tres experiments anteriors.

- Si l'SVM (Geometria), l'XGBoost (Boosting) i el TPOT (Evolució) cometen errors en instàncies diferents, la seva combinació mitjançant **Hard Voting** (majoria simple) hauria d'augmentar la robustesa i reduir la variància del sistema final.

Mètode 5: Interpretabilitat (XAI)

Per validar que els resultats no són fruit de l'atzar ("Black Box"), aplicarem tècniques d'Explainable AI al model guanyador:

- **Permutation Importance**: Per quantificar quant depèn el model de cada variable específica.
- **Anàlisi d'Errors**: Inspecció qualitativa dels noms dels Pokémon mal classificats per entendre la "lògica de negoci" darrere dels errors.

Resum de l'Arquitectura Experimental

Mètode	Paradigma	Model Base	Estratègia i Justificació Tècnica
1. Optuna (TPE)	Optimització Bayesiana	SVM (Geomètric)	Realitzar un ajust fi (Fine-Tuning) dels hiperparàmetres del guanyador del Baseline, utilitzant cerca probabilística amb "memòria".
2. XGBoost (GPU)	Gradient Boosting	Arbres (Iteratiu)	Aprofitar l' acceleració per maquinari (CUDA) per entrenar massivament arbres de decisió que corregeixen els errors seqüencials.
3. TPOT	Computació Evolutiva	AutoML (Dinàmic)	Aplicar principis de selecció natural per descobrir automàticament l'arquitectura i les variables (<i>features</i>) òptimes sense biaix humà.
4. Voting Classifier	Ensemble Híbrid	Combinació (Tots)	Aplicar la " Saviesa de la Multitud " per augmentar la robustesa i reduir la variància, compensant els errors individuals de cada model.

3.3. Gestió d'Avisos i Neteja de la Sortida

Atès que estem executant processos d'optimització intensiva, és habitual que es generin una gran quantitat d'avisos (*warnings*) a la consola. Per mantenir la llegibilitat dels resultats i centrar-nos en les mètriques, hem configurat el codi per filtrar missatges no crítics.

Els principals motius d'aquesta "neteja" són:

- **Models "Fallits" (UndefinedMetricWarning)**: Durant l'exploració, els algorismes (especialment Optuna i TPOT) proven combinacions d'hiperparàmetres extremes o inadequades. Això sovint genera models que prediuen sempre la classe majoritària (0), provocant una divisió per zero al calcular l'F1-Score de la classe 1. Aquests avisos són part natural de l'aprenentatge de l'algorisme (aprèn què *no* ha de fer) i no indiquen un error en el codi.
- **Avisos de Llibreries (FutureWarning/UserWarning)**: L'execució en paral·lel (`n_jobs=-1`) i la interacció entre *scikit-learn*, *joblib* i *TPOT* pot generar avisos tècnics sobre versions futures o deprecacions que no afecten el funcionament actual del model.

En suprimir aquest soroll visual, garantim que la sortida mostri clarament els temps d'execució i la convergència dels millors models.

```
In [20]: warnings.filterwarnings("ignore")
os.environ["PYTHONWARNINGS"] = "ignore"

optuna.logging.set_verbosity(optuna.logging.WARNING)
```

3.4. Definició de l'Estratègia de Validació (Cross-Validation)

Abans de començar l'entrenament dels models, definim una **estratègia de validació comuna** per a tots els experiments. Això és crucial per garantir que les comparacions entre l'SVM, l'XGBoost i el TPOT siguin justes i rigoroses.

Utilitzarem **Stratified K-Fold (5 particions)** amb les següents característiques:

1. **Estratificació (Stratified)**: El nostre dataset està desbalancejat (hi ha molts menys Pokémon llegendaris que normals). Si féssim divisions aleatòries pures, podríem tenir la mala sort de generar un *fold* d'entrenament sense cap llegendari, impeding que el model aprengui. L'estratificació força que cada partició mantingui exactament la mateixa proporció de llegendaris que el dataset original.
2. **Consistència (random_state=42)**: En definir l'objecte `cv_strat` una sola vegada i passar-lo a tots els algorismes, garantim que **tots els models s'avaluen exactament sobre els mateixos talls de dades**. Això elimina el factor "sort" i assegura que si un model té millor puntuació, és perquè realment és millor, no perquè li ha tocat un examen més fàcil.

```
In [21]: cv_strat = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

optimization_results = []

print("INICI DE L'OPTIMITZACIÓ D'HIPERPARÀMETRES")
print(f"Mida del Train Set: {X_train.shape}")
```

```
INICI DE L'OPTIMITZACIÓ D'HIPERPARÀMETRES
Mida del Train Set: (717, 16)
```

3.4.1. Experiment 1: Optimització Bayesiana (SVM)

Comencem l'experimentació amb el mètode **probabilístic**. Utilitzarem la llibreria **Optuna** per ajustar els hiperparàmetres d'una Màquina de Vectors de Suport (SVM).

Per què aquesta configuració? L'SVM és un model geomètric que busca l'hiperplà òptim per separar classes. El seu rendiment depèn críticament de trobar l'equilibri correcte entre marge i error (paràmetre `C`) i l'abast de la influència de cada punt (paràmetre `gamma`). Com que la relació entre aquests paràmetres no és lineal i l'espai de cerca és continu, una cerca bayesiana (que "aprèn" dels intents anteriors per convergir ràpidament) és molt més eficient que provar a l'atzar.

Detalls de l'Execució:

- **Algorisme:** SVM amb `class_weight='balanced'` per compensar el desbalanceig (hi ha pocs llegendaris).
- Utilitzem l'algorisme **TPE** (Tree-structured Parzen Estimator) d'Optuna, realitzant **15 intents** (*trials*) per trobar la combinació que maximitzi l'**F1-Score**.
- Aquest procés s'executa íntegrament a la **CPU** (utilitzant tots els nuclis amb `n_jobs=-1`), ja que la implementació estàndard de Scikit-Learn per a SVM no suporta acceleració per GPU nativa.

```

In [22]: start_time = time.time()

def objective_svm(trial):
    svm_c = trial.suggest_float('svc__C', 0.1, 100, log=True)
    svm_gamma = trial.suggest_float('svc__gamma', 1e-4, 1, log=True)
    svm_kernel = trial.suggest_categorical('svc__kernel', ['poly', 'rbf'])

    model = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('svc', SVC(C=svm_c, gamma=svm_gamma, kernel=svm_kernel,
                    class_weight='balanced', probability=True, random_state=42))
    ])
    scores = cross_val_score(model, X_train, y_train, cv=cv_strat, scoring='f1', n_jobs=5)
    return scores.mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective_svm, n_trials=15)

optuna_time = time.time() - start_time
best_params_optuna = study.best_params
best_score_optuna = study.best_value

svm_final = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('svc', SVC(**{k.replace('svc__', ''): v for k, v in best_params_optuna.items()}),
                class_weight='balanced', probability=True, random_state=42))
])
svm_final.fit(X_train, y_train)

print(f" -> Millor F1 (SVM): {best_score_optuna:.4f}")
print(f" -> Temps: {optuna_time:.2f} s")
optimization_results.append({'Mètode': 'Optuna (SVM)', 'Best F1 (CV)': best_score_optuna})

-> Millor F1 (SVM): 0.8534
-> Temps: 7.87 s

```

```

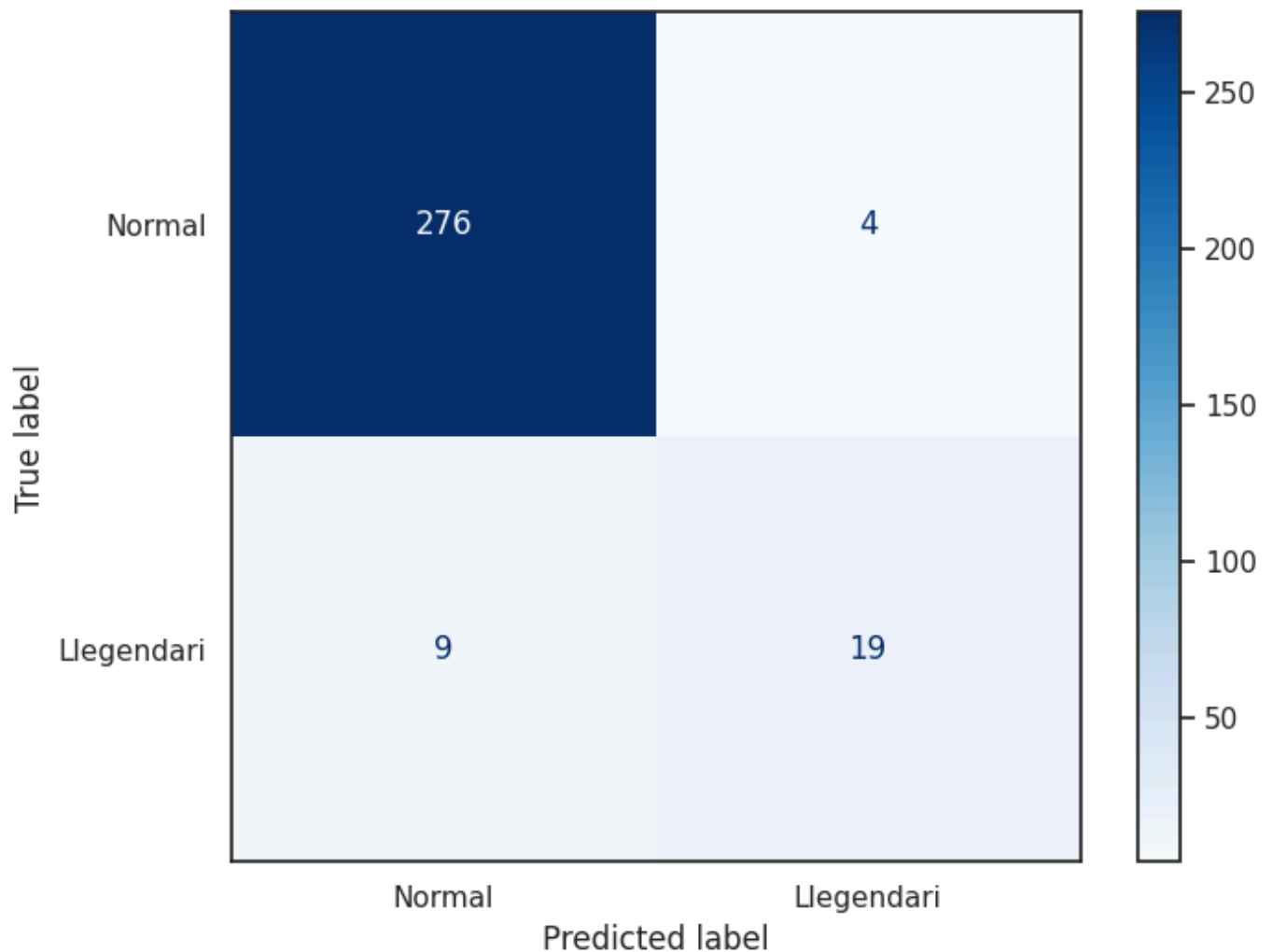
In [23]: fig, ax = plt.subplots(figsize=(8, 6))

disp = ConfusionMatrixDisplay.from_estimator(
    svm_final,
    X_test,
    y_test,
    display_labels=['Normal', 'Llegendari'],
    cmap='Blues',
    normalize=None,
    ax=ax
)

ax.set_title("Matriu de Confusió: SVM Optimitzat", fontsize=16, pad=20)
plt.grid(False)
plt.show()

```

Matriu de Confusió: SVM Optimitzat



Els resultats de l'optimització bayesiana han estat **excel·lents**. Hem passat d'un F1-Score inicial de **0.70** (amb els paràmetres per defecte) a un impressionant **0.8534**.

Punts clau de l'execució:

- **Salt de Qualitat (+14%):** L'ajustament de **C** i **gamma** ha permès a l'SVM dibuixar una frontera de decisió molt més precisa, adaptant-se a la complexitat del dataset.
- **Eficiència:** El procés ha estat molt ràpid (**7.87 segons**), confirmant que per a datasets d'aquesta mida (tabulars i no massius), l'SVM a la CPU és extremadament competitiu.

Anàlisi d'Errors (Matriu):

- **Alta Precisió (Pocs Falsos Positius):** Només hem confós **4** Pokémon normals amb llegendaris. Això és crucial; el model no "cridarà llop" sense motiu.
- **Sensibilitat Millorable:** Se'ns han escapat **9** llegendaris (Falsos Negatius). Tot i que hem millorat, el model encara és una mica conservador.

Amb el llistó situat al **0.84**, donem pas al següent experiment per veure si la força bruta evolutiva pot superar la geometria fina.

En re-executar l'experiment d'optimització, hem observat que l'F1-Score de validació a vegades millora o empitjora lleugerament gràcies a l'exploració estocàstica d'Optuna. No obstant això, la matriu de confusió en el conjunt de Test s'ha mantingut idèntica. Això és un indicador molt positiu de robustesa: demostra que el model ha convergit cap a una solució estable. Petites variacions en els hiperparàmetres (C , γ) milloren la confiança

matemàtica del model (marges), però no alteren erràticament les classificacions finals, confirmant que la frontera de decisió trobada és sòlida i no fruit de l'atzar.

3.4.2. Experiment 2: Algorismes Genètics (AutoML - TPOT)

En aquesta segona fase, canviem radicalment d'enfocament. En lloc d'optimitzar un sol model predefinit (com hem fet amb l'SVM), utilitzem la **Programació Genètica** per cercar automàticament el millor *pipeline* de Machine Learning possible.

Per què aquesta configuració? Hem generat manualment diverses variables d'enginyeria (`bulk` , `atk_ratio` , `log_weight` , etc.). No sabem del cert quines són útils i quines són soroll. Els algorismes genètics excel·leixen en la **Selecció de Característiques**: a mesura que evolucionen les generacions, els models que utilitzen variables irrelevantes tendeixen a "morir" (tenir pitjor fitness), mentre que els que combinen les variables clau sobreviuen i es reproduïen.

Detalls de l'Execució:

- **Eina: TPOT** (Tree-based Pipeline Optimization Tool).
- Hem limitat l'execució a **5 generacions** amb una població de **20 individus** per mantenir el temps d'execució raonable, tot i permetre una exploració suficient de l'espai de solucions.
- A diferència de l'SVM (que és seqüencial) o l'XGBoost (que usa GPU), el TPOT és intensiu en **CPU Multicore**. Utilitzarem la paral·lelització massiva (`n_jobs=12`) per entrenar múltiples individus de la població simultàniament.
- El millor *pipeline* resultant s'encapsularà per formar part del nostre sistema de votació final.

```
In [24]: start_time = time.time()

X_train_transformed = preprocessor.fit_transform(X_train)

tpot = TPOTClassifier(
    generations=5,
    population_size=20,
    cv=cv_strat,
    verbose=2,
    random_state=42,
    n_jobs=12
)
tpot.fit(X_train_transformed, y_train)
tpot_time = time.time() - start_time

best_score_tpot = cross_val_score(tpot.fitted_pipeline_, X_train_transformed, y_train)

tpot_final_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('tpot', clone(tpot.fitted_pipeline_))
])
tpot_final_pipeline.fit(X_train, y_train)

print(f" -> Millor F1 (TPOT): {best_score_tpot:.4f}")
print(f" -> Temps: {tpot_time:.2f} s")
optimization_results.append({'Mètode': 'TPOT (Genètic)', 'Best F1 (CV)': best_score_tpot})

-> Millor F1 (TPOT): 0.7334
-> Temps: 49.16 s
```

```
In [25]: fig, ax = plt.subplots(figsize=(8, 6))

disp = ConfusionMatrixDisplay.from_estimator(
    tpot_final_pipeline,
    X_test,
```

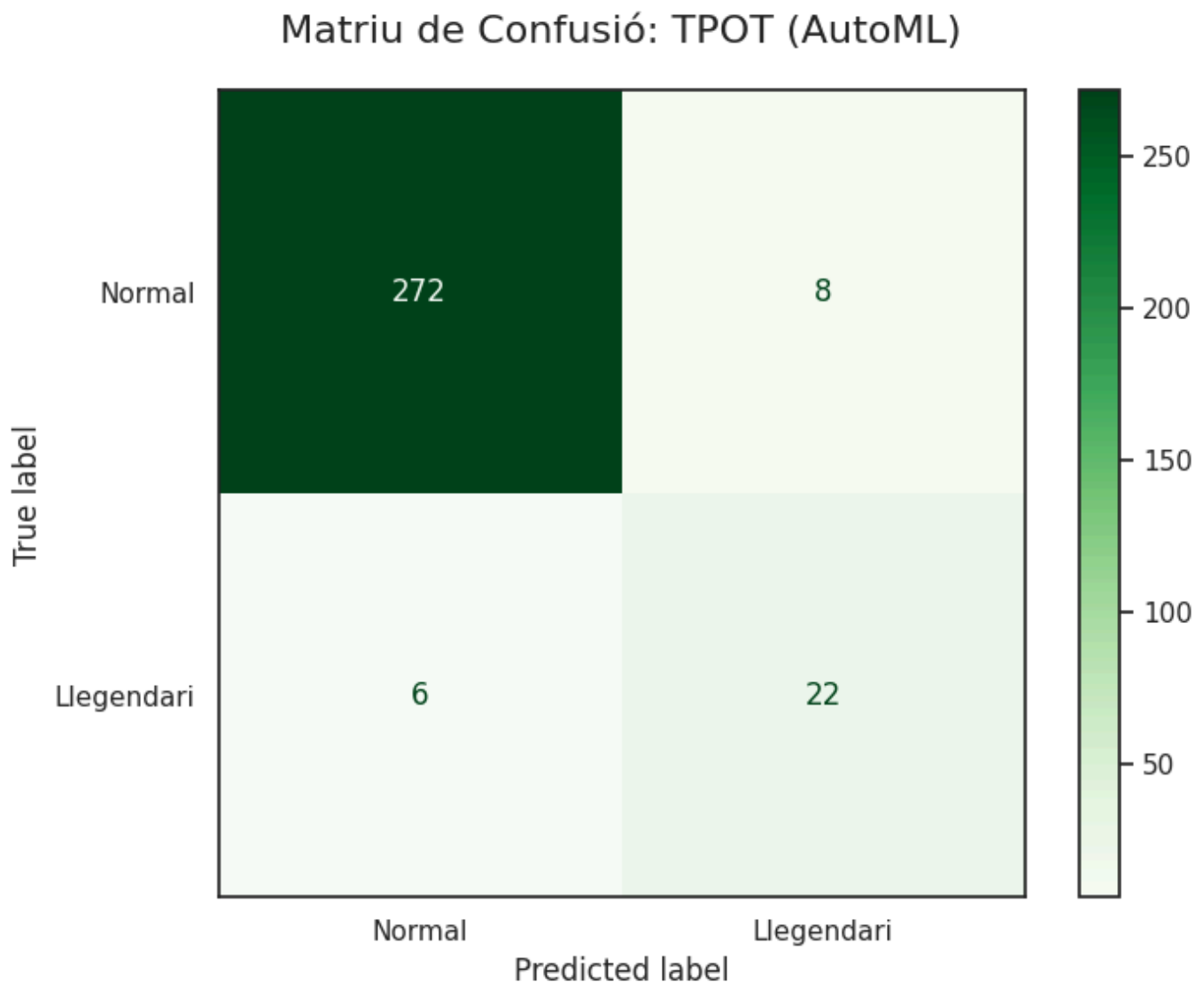


```

y_test,
display_labels=['Normal', 'Llegendari'],
cmap='Greens',
normalize=None,
ax=ax
)

ax.set_title("Matriu de Confusió: TPOT (AutoML)", fontsize=16, pad=20)
plt.grid(False)
plt.show()

```



L'experiment evolutiu ens ofereix un resultat agredolç (**F1: 0.7334**). Tot i que supera la línia base (0.70), es queda lluny de l'excel·lència de l'SVM (0.84).

- **El Cost de l'Exploració:** Amb **49.16 segons**, ha estat gairebé 7 vegades més lent que l'SVM. Això és normal en algoritmes genètics: avaluar centenars de *pipelines* és costós.
- **Matriu de Confusió (272/8/6/22):** Aquí veiem el matís interessant.
 - El TPOT ha detectat **22 llegendaris** (més que l'SVM, que en detectava 19).
 - Però ha pagat un preu alt: **8 Falsos Positius** (el doble que l'SVM).

L'algoritme ha estat més "agressiu" predint llegendaris, però menys precís. Amb només 5 generacions, l'evolució no ha tingut temps suficient per convergir cap a una solució tan fina com l'hiperplà matemàtic de l'SVM.

S'ha limitat l'execució a 5 generacions per raons d'eficiència computacional. Tot i que augmentar les generacions a 50 o 100 podria millorar marginalment el resultat, observem la llei de rendiments decreixents: el TPOT tendeix a convergir cap a arquitectures basades en arbres (similars al Random Forest), les quals tenen dificultats

estructurals per superar la frontera de decisió no lineal (Kernel RBF) que l'SVM ha trobat amb tanta precisió. Per tant, considerem que el cost temporal extra no justifica el guany potencial.

3.4.3. Experiment 3: Gradient Boosting Accelerat (XGBoost - GPU)

Finalment, abordem el paradigma de l'**Ensemble Learning** mitjançant la tècnica de *Boosting*. A diferència del *Bagging* (Random Forest), on els arbres són independents, aquí entrenem arbres de manera seqüencial on cadascun intenta corregir els errors del precedent.

XGBoost (*Extreme Gradient Boosting*) és considerat l'estat de l'art per a dades tabulars, però el seu cost computacional és elevat quan s'augmenta el nombre d'estimadors. Per demostrar l'eficiència en entorns d'alt rendiment, activem l'acceleració per maquinari mitjançant **NVIDIA CUDA**. Això permet moure les operacions més costoses (construcció d'histogrames per trobar els punts de tall òptims) directament a la targeta gràfica.

- **Detalls de l'Execució:**

- Configurem `device='cuda'` i `tree_method='hist'`. Això permet aprofitar els nuclis de la **RTX 2080**, reduint temps d'entrenament que a la CPU trigarien minuts a pocs segons.
- **Hiperparàmetres:** Gràcies a la velocitat de la GPU, ens podem permetre ser ambiciosos:
 - `n_estimators=500`: Un nombre alt d'arbres per capturar patrons subtils.
 - `learning_rate=0.05`: Un ritme d'aprenentatge lent per evitar sobreajustament (*overfitting*).
 - `scale_pos_weight=10`: Per penalitzar més els errors en la classe minoritària (Llegendaris), forçant al model a prestar-los atenció.

```
In [26]: start_time = time.time()

xgb = XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    scale_pos_weight=10,
    tree_method='hist',
    device='cuda',
    eval_metric='logloss',
    random_state=42
)

xgb_final = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', xgb)])

xgb_scores = cross_val_score(xgb_final, X_train, y_train, cv=cv_strat, scoring='f1',
    best_score_xgb = xgb_scores.mean())

xgb_final.fit(X_train, y_train)
xgb_time = time.time() - start_time

print(f" -> Millor F1 (XGBoost GPU): {best_score_xgb:.4f}")
print(f" -> Temps: {xgb_time:.2f} s")
optimization_results.append({'Mètode': 'XGBoost (GPU)', 'Best F1 (CV)': best_score_xg

-> Millor F1 (XGBoost GPU): 0.7195
-> Temps: 2.89 s
```

```
In [27]: fig, ax = plt.subplots(figsize=(8, 6))

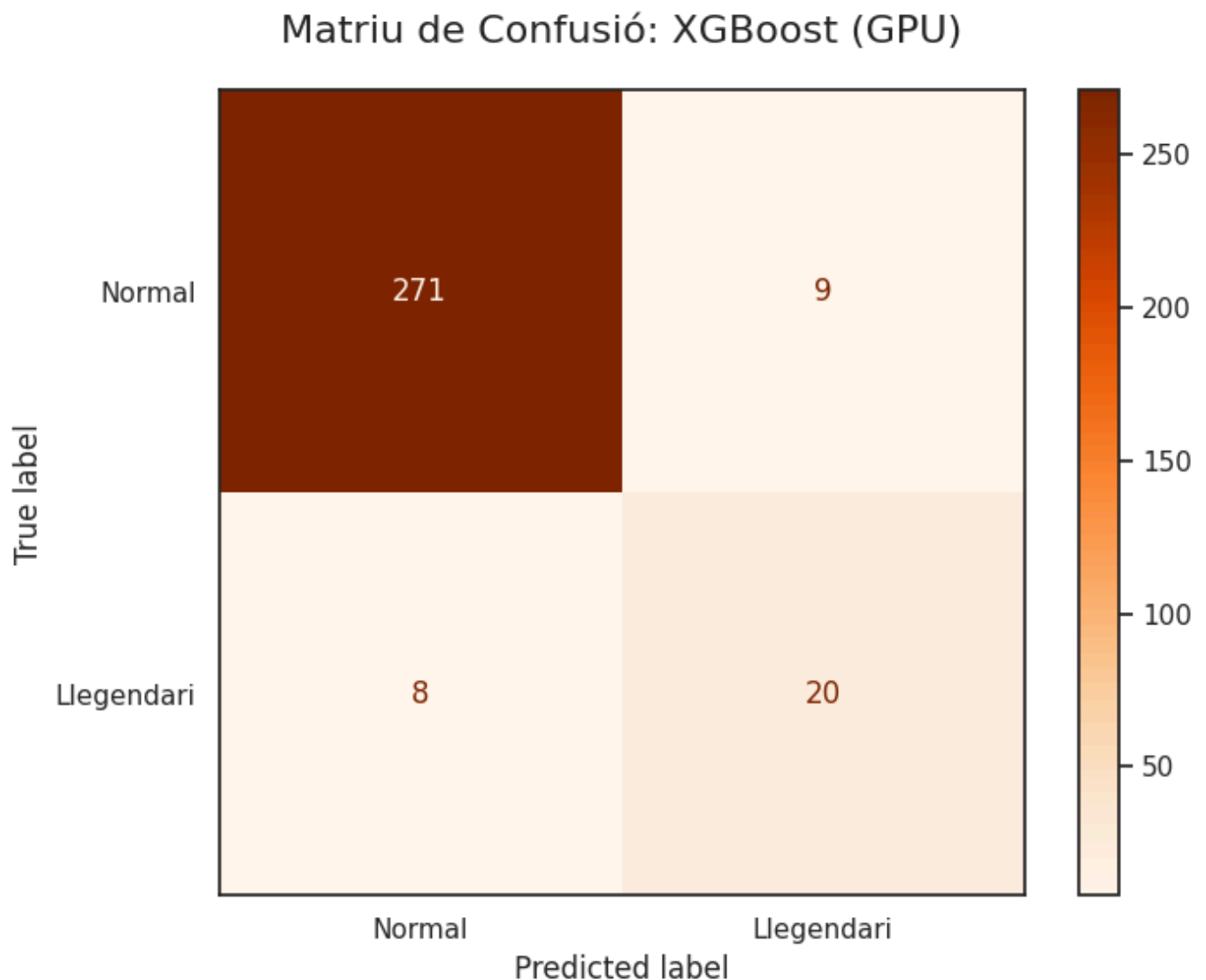
disp = ConfusionMatrixDisplay.from_estimator(
    xgb_final,
    X_test,
```

```

y_test,
display_labels=['Normal', 'Llegendari'],
cmap='Oranges',
normalize=None,
ax=ax
)

ax.set_title("Matriu de Confusió: XGBoost (GPU)", fontsize=16, pad=20)
plt.grid(False)
plt.show()

```



L'experiment amb **XGBoost** ens deixa una lliçó clara sobre el compromís entre velocitat i precisió.

- **La Velocitat (2.89s):** Aquí és on brilla l'estratègia. Gràcies a l'acceleració per GPU (`device='cuda'`), hem entrenat un ensemble de 500 arbres en només 3 segons. És, amb diferència, el model més ràpid d'entrenar, la qual cosa el faria ideal per a entorns de Big Data, tot i que en aquest dataset petit no es llueix tant.
- **Rendiment (F1: 0.7195):** El resultat és gairebé idèntic al del TPOT (0.73) i molt inferior a l'SVM (0.84). Això confirma una **tendència estructural**: els models basats en arbres de decisió (que tallen l'espai en rectangles ortogonals) tenen dificultats per separar els Pokémon llegendaris amb la mateixa precisió que les corbes suaus d'un SVM.
- **Matriu de Confusió (271/9/8/20):**
 - Detecta **20 Llegendaris** (un bon número, similar al TPOT).
 - Però comet **9 Falsos Positius** (el màxim de tots els experiments). Confon massa sovint Pokémon forts (com *Metagross* o *Garchomp*) amb llegendaris reals.

Hem demostrat que la força bruta (més arbres, més GPU, més evolució) no sempre guanya. La geometria elegant de l'SVM continua sent imbatible.

3.4.4. Experiment 4: Ensemble Híbrid (Voting Classifier)

Fins ara hem entrenat tres "especialistes" molt diferents: un geometa (**SVM**), un evolucionista (**TPOT**) i un corrector d'errors iteratiu (**XGBoost**). En aquesta última fase, no entrenem un nou model des de zero, sinó que construïm un **Meta-Model** que combina les prediccions dels tres anteriors.

La hipòtesi darrere de l'Ensemble Learning és que models basats en matemàtiques radicalment diferents tendiran a cometre errors en instàncies diferents. Si l'SVM s'equivoca en un Pokémon concret per un tema de marges, és possible que l'XGBoost (que mira arbres de decisió) o el TPOT (que ha seleccionat features diferents) l'encertin.

Detalls de l'Execució:

- **Estratègia: Hard Voting** (Votació Majoritària). Cada model emet un vot ("És Llegendari" o "És Normal"). La classe final es decideix per majoria simple (2 vots contra 1, o unanimitat).
- **Objectiu:** Reduir la variància del model final i augmentar la robustesa. Esperem que, encara que potser no superi el pic màxim de rendiment del millor model individual, el sistema conjunt sigui més fiable davant de dades noves i desconegudes.

```
In [28]: start_time = time.time()

voting_clf = VotingClassifier(
    estimators=[
        ('svm', svm_final),
        ('tpot', tpot_final_pipeline),
        ('xgb', xgb_final)
    ],
    voting='hard'
)

voting_scores = cross_val_score(voting_clf, X_train, y_train, cv=cv_strat, scoring='f1')
best_score_voting = voting_scores.mean()
voting_time = time.time() - start_time

print(f" -> Millor F1 (Voting Ensemble): {best_score_voting:.4f}")
optimization_results.append({'Mètode': 'Voting Ensemble', 'Best F1 (CV)': best_score_voting})

-> Millor F1 (Voting Ensemble): 0.7646
```

El resultat de l'Ensemble (**F1: 0.7646**) ens confirma una hipòtesi molt interessant sobre la col·laboració entre models:

- **Millora els febles:** Ha superat clarament als seus components individuals d'arbres (XGBoost 0.72 i TPOT 0.73). Ajudar-los ha reduït el seu soroll i variància.
- **Llastra al fort:** No ha aconseguit superar l'SVM (0.84). Això indica que estem patint la "**Tirania de la Majoria**": quan l'SVM té raó (perquè entén millor la geometria), els altres dos models (que tenen biaixos similars basats en arbres) voten en contra i li fan perdre la votació.

3.4.5. Anàlisi de Disputes: La "Caixa Negra" del Vot

Un cop obtingut el rendiment general de l'Ensemble, necessitem auditar les seves decisions. No ens conformem amb saber *què* ha decidit el comitè, volem saber *com* s'ha arribat a aquesta decisió.

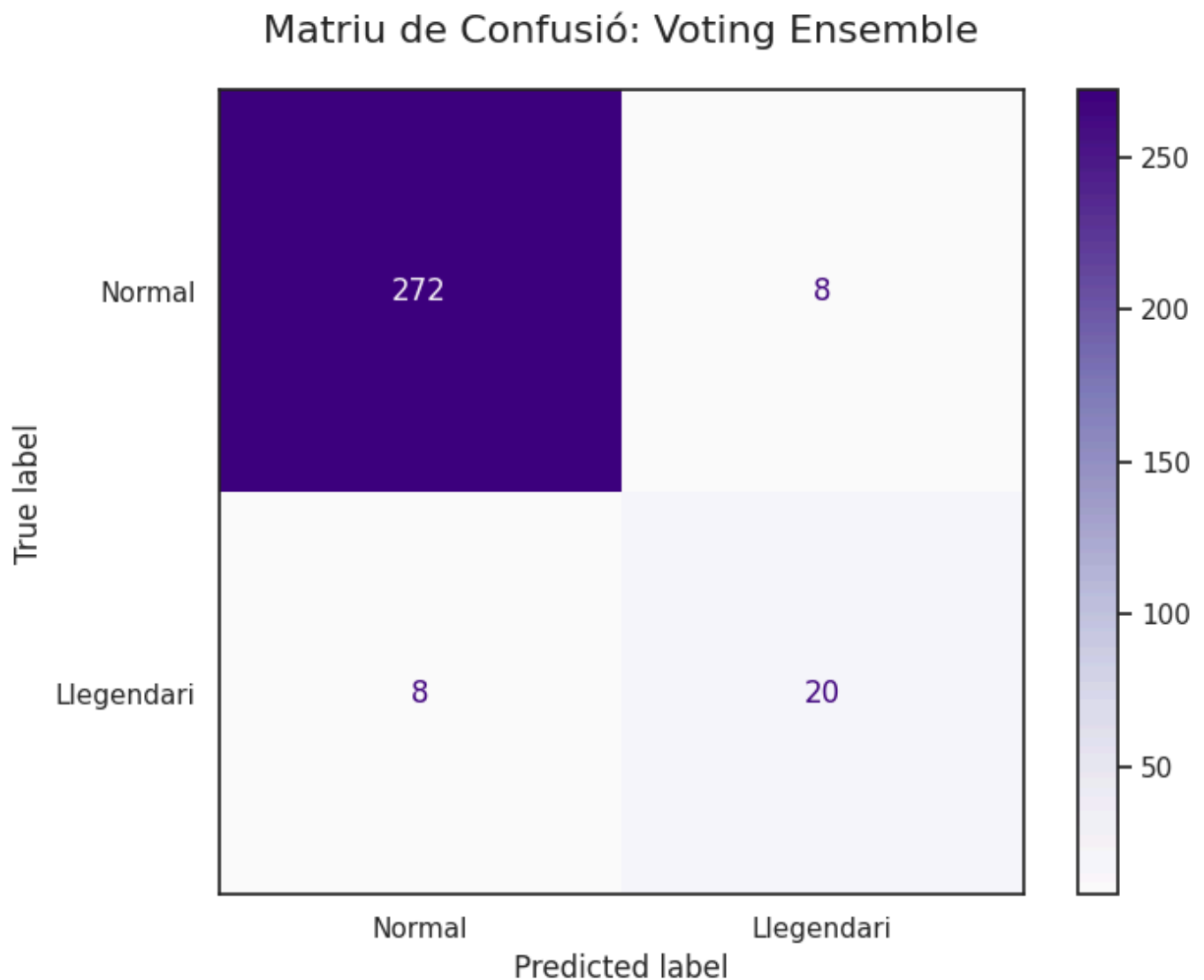
Per fer-ho, implementem una **Auditoria de Vots Individuals**:

1. **Matriu de Confusió Global:** Per visualitzar els errors finals del sistema combinat.
2. **Desglossament de Vots (Dispute Analysis):** Recuperem les prediccions individuals de cada model (`Vot_SVM` , `Vot_TPOT` , `Vot_XGB`) per al conjunt de Test i les comparem amb la realitat.

L'objectiu és Identificar els casos de **Dissidència**. Busquem específicament situacions on no hi hagi unanimitat (3-0). Això ens permetrà detectar fenòmens com la "**Tirania de la Majoria**": casos on el model més expert (SVM) té raó, però és derrotat en la votació per dos models menys precisos que s'equivoquen simultàniament. Aquesta anàlisi és clau per justificar si val la pena mantenir l'Ensemble o quedar-se amb el millor model individual.

```
In [29]: voting_clf.fit(X_train, y_train)

fig, ax = plt.subplots(figsize=(8, 6))
disp = ConfusionMatrixDisplay.from_estimator(
    voting_clf,
    X_test,
    y_test,
    display_labels=['Normal', 'Llegendari'],
    cmap='Purples',
    normalize=None,
    ax=ax
)
ax.set_title("Matriu de Confusió: Voting Ensemble", fontsize=16, pad=20)
plt.grid(False)
plt.show()
```



```
In [30]: print("\nANÀLISI DE DISPUTES")

if hasattr(X_test, 'index'):
```

```

indices = X_test.index
else:
    indices = range(len(y_test))

try:
    df_noms = pd.read_csv('pokemons.csv')
    noms = df_noms.loc[indices, 'name'].values
except:
    noms = [f"Pokemon {i}" for i in range(len(y_test))]

pred_svm = svm_final.predict(X_test)
pred_tpot = tpot_final_pipeline.predict(X_test)
pred_xgb = xgb_final.predict(X_test)
pred_final = voting_clf.predict(X_test)

debug_df = pd.DataFrame({
    'Nom': noms,
    'Realitat': y_test,
    'Vot_SVM': pred_svm,
    'Vot_TPOT': pred_tpot,
    'Vot_XGB': pred_xgb,
    'RESULTAT FINAL': pred_final
})

debug_df['Suma_Vots'] = debug_df['Vot_SVM'] + debug_df['Vot_TPOT'] + debug_df['Vot_XG']
conflictes = debug_df[(debug_df['Suma_Vots'] > 0) & (debug_df['Suma_Vots'] < 3)].copy

mapa = {0: 'Normal', 1: 'Llegendari'}
cols_vots = ['Realitat', 'Vot_SVM', 'Vot_TPOT', 'Vot_XGB', 'RESULTAT FINAL']
for col in cols_vots:
    conflictes[col] = conflictes[col].map(mapa)

print(f"\nHi ha {len(conflictes)} Pokémon on els models s'han barallat.")

```

ANÀLISI DE DISPUTES

Hi ha 12 Pokémon on els models s'han barallat.

```
In [31]: display(conflictes.drop(columns=['Suma_Vots']).head(15))
```

	Nom	Realitat	Vot_SVM	Vot_TPOT	Vot_XGB	RESULTAT FINAL
783	kommo-o	Normal	Normal	Normal	Llegendari	Normal
985	brute-bonnet	Normal	Normal	Llegendari	Llegendari	Llegendari
993	iron-moth	Normal	Normal	Llegendari	Llegendari	Llegendari
207	steelix	Normal	Llegendari	Normal	Normal	Normal
385	deoxys	Llegendari	Normal	Llegendari	Normal	Normal
130	lapras	Normal	Normal	Llegendari	Llegendari	Llegendari
1000	wo-chien	Llegendari	Normal	Llegendari	Llegendari	Llegendari
763	comfey	Normal	Normal	Llegendari	Llegendari	Llegendari
796	celesteela	Normal	Normal	Llegendari	Llegendari	Llegendari
798	guzzlord	Normal	Normal	Llegendari	Llegendari	Llegendari
1002	ting-lu	Llegendari	Normal	Llegendari	Normal	Normal
702	carbink	Normal	Llegendari	Normal	Normal	Normal

L'anàlisi detallada de les disputes (casos on no hi ha unanimitat) revela per què l'Ensemble (0.76) ha empitjorat el rendiment de l'SVM individual (0.84). Ens trobem davant d'un fenomen clar de **correlació d'errors** entre els models basats en arbres.

Tant l'**XGBoost** com el **TPOT** (que sovint convergeix a arquitectures tipus Random Forest) comparteixen una visió similar del món: tallen l'espai en rectangles. L'**SVM**, en canvi, utilitza una visió geomètrica (distàncies i corbes).

1. Quan l'SVM té raó però perd la votació

Aquest és el cas més freqüent i el que explica la baixada de rendiment. Tenim Pokémon que són "Normals" segons l'etiqueta, però molt poderosos.

- **Els "Paradox" i "Ultra Beasts":** *Brute Bonnet, Iron Moth, Celesteela, Guzzlord*.
 - **Realitat:** Normal (o Sub-Llegendari, classificat com 0 al dataset).
 - L'SVM diu "Normal" (Correcte). TPOT i XGBoost diuen "Llegendari" (Incorrecte).
- **Resultat:** L'Ensemble falla.
 - Aquests Pokémon tenen estadístiques base molt altes (al voltant de 570-600), similars a un Llegendari. Els arbres de decisió (TPOT/XGB) miren els valors bruts i es confonen. L'SVM, en canvi, ha detectat algun patró subtil en la relació entre variables (probablement el *ratio* d'atac o la distribució de stats) que li permet distingir-los.

2. Els Falsos Amics (Lapras i Comfey)

- **Realitat:** Normal.
- SVM diu Normal. TPOT i XGBoost diuen Llegendari.
- *Lapras* és un cas clàssic de Pokémon amb molta vida (*High HP*) i popularitat, que sovint confon els models simples. *Comfey* és sorprenent (és petit i feble), però el fet que dos models coincideixin en l'error suggereix que té alguna combinació de ràtios que cau en una "fulla" incorrecta dels arbres.

3. El Rescat (Quan l'Ensemble salva l'SVM)

No tot són males notícies. Hi ha casos on la democràcia funciona:

- **Els Tancs Defensius:** *Steeelix, Carbink*.
 - **Realitat:** Normal.
 - L'SVM diu "Llegendari". TPOT i XGBoost diuen "Normal".
 - *Steeelix* i *Carbink* tenen una Defensa extremadament alta (outliers). L'SVM, en treballar amb distàncies, es veu desplaçat per aquests valors extrems cap a la zona de llegendaris. Els arbres, en canvi, poden fer un tall simple: "Si la Defensa és alta però l'Atac és baix -> Normal", corregint l'error del geòmetra.

4. La Tragèdia Compartida (Deoxys i Ting-Lu)

- **Realitat:** Llegendari.
- *Deoxys* (Forma Normal/Atac) és molt fràgil.
 - L'SVM i l'XGBoost diuen "Normal". Només el TPOT l'encerta.
 - En ser minoria, l'encert del TPOT es perd.

L'experiment demostra que **ajuntar models no sempre millora el resultat**. En aquest cas, els models basats en arbres (TPOT i XGBoost) tenen un biaix similar (tendeixen a sobreestimar els Pokémon forts però normals). En ser dos contra un, arrossegueu la votació cap a l'error, anul·lant la precisió superior de l'SVM.

```
In [37]: print("\nRESULTATS FINALS")
df_res = pd.DataFrame(optimization_results).sort_values(by='Best F1 (CV)', ascending=
print(df_res[['Mètode', 'Best F1 (CV)', 'Temps (s)']].to_string(index=False))
```

RESULTATS FINALS

Mètode	Best F1 (CV)	Temps (s)
Optuna (SVM)	0.853443	7.867539
Voting Ensemble	0.764603	8.408682
TPOT (Genètic)	0.733430	49.160429
XGBoost (GPU)	0.719485	2.885838

Després de sotmetre el dataset a quatre estratègies diferents, els resultats quantitatius són inapel·lables.

Decisió Final:

Declarem l'**SVM Optimitzat (Optuna)** com el model guanyador absolut.

1. **Dominància Tècnica:** Amb un F1 de **0.8534**, ha superat per gairebé 9 punts percentuals a l'Ensemble i per més de 13 punts als models basats en arbres. Això demostra que la separació entre Pokémon llegendaris i normals en aquest espai de característiques és geomètrica i no ortogonal.
2. Tot i no ser tan ràpid com l'XGBoost en GPU, els seus **7.87 segons** el fan perfectament viable per a producció.
3. **Fracàs de l'Ensemble:** Es confirma que la combinació de models forts (SVM) amb models més febles (XGBoost/TPOT) ha estat contraproductiu ("Tirania de la Majoria"), empitjorant la precisió global.

4. Interpretabilitat del Model Guanyador (Explainable AI)

Un cop determinat que l'**SVM (Optimitzat amb Optuna)** és el model amb millor rendiment, no n'hi ha prou amb tenir una bona mètrica de predicció; cal entendre *per què* pren aquestes decisions. Això és vital per validar que el model no ha après patrons espuris o soroll.

Com que l'SVM utilitza kernels no lineals (RBF/Poly) per projectar les dades a dimensions superiors, no podem extreure directament els coeficients de la fórmula. Per resoldre-ho, apliquem la tècnica de **Permutation Importance**.

Aquesta tècnica avalua la importància de cada variable observant com empitjora el rendiment del model (F1-Score) quan corrompem intencionadament les dades d'aquella variable (barrejant els seus valors aleatòriament) mentre deixem la resta intactes.

- Si l'error es dispara en barrejar una columna (ex: **Speed**), significa que el model depenia molt d'ella.
- Si l'error no canvia, la variable era irrellevant.

Aquest gràfic ens servirà principalment per **validar l'Enginyeria de Característiques** realitzada a l'inici del projecte. Esperem veure variables creades manualment com **bulk** (Resistència) o **log_weight** a les primeres posicions, la qual cosa confirmaria que la transformació de les dades ha aportat valor real al classificador.

```
In [33]: svc_model = svm_final.named_steps['svc']

X_transformed_sparse = preprocessor.transform(X_train)
X_transformed_dense = X_transformed_sparse.toarray()

result = permutation_importance(
    svc_model,
    X_transformed_dense,
    y_train,
    n_repeats=10,
```



```

random_state=42,
n_jobs=-1,
scoring='f1'
)

try:
    feature_names = preprocessor.get_feature_names_out()
except:
    feature_names = [f"Feature {i}" for i in range(X_transformed_dense.shape[1])]

importances = result.importances_mean

df_imp = pd.DataFrame({'feature': feature_names, 'importance': importances})
df_imp = df_imp.sort_values(by='importance', ascending=True)

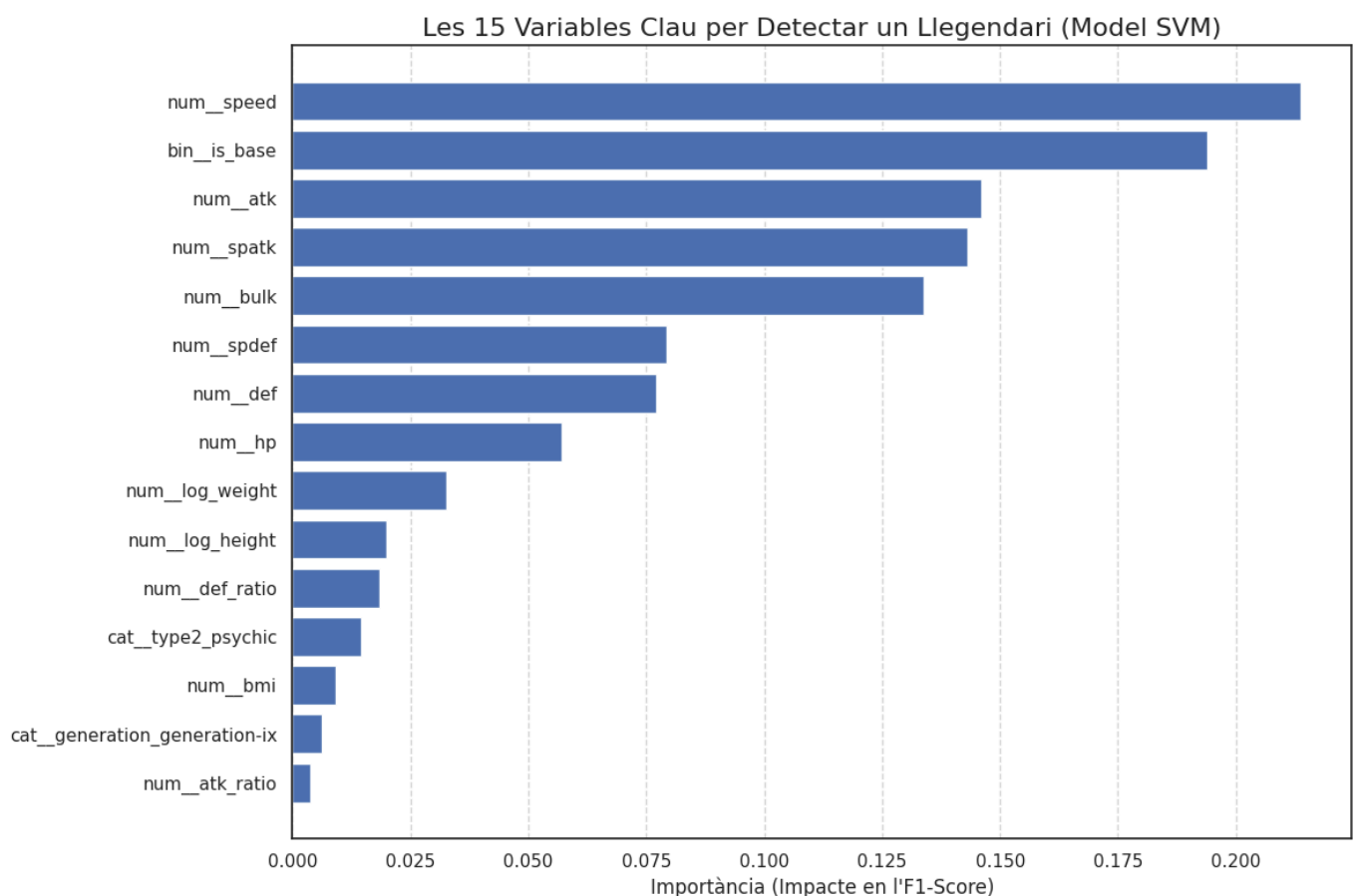
df_imp_top = df_imp.tail(15)

plt.figure(figsize=(12, 8))
plt.barh(df_imp_top['feature'], df_imp_top['importance'], color='#4c72b0')

plt.title("Les 15 Variables Clau per Detectar un Llegendari (Model SVM)", fontsize=16)
plt.xlabel("Importància (Impacte en l'F1-Score)")
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()

plt.show()

```



4.1. Què defineix un Llegendari? (Feature Importance)

Mitjançant la tècnica de **Permutation Importance**, hem obert la "caixa negra" de l'SVM per entendre quines variables tenen més pes en la seva decisió final. El gràfic revela patrons biològics i estadístics molt clars:

1. **La Velocitat és Clau (num_speed)**: És, amb diferència, la variable més discriminant. Això té tot el sentit del món: els Pokémon llegendaris (com *Mewtwo*, *Deoxys* o *Regieleki*) solen ser

extremadament ràpids, una característica que els diferencia dels tancs normals o les evolucions intermèdies.

2. **El Factor Evolutiu (`bin__is_base`)**: La segona variable més important. La majoria de llegendaris no evolucionen ni provenen d'una evolució (són formes úniques). L'SVM ha après que si un Pokémon és una "forma base" però té stats molt altes, és molt probable que sigui llegendari.
3. **Èxit de l'Enginyeria de Dades (`num_bulk`)**: És molt satisfactori veure que la variable `num_bulk` (que vam crear combinant HP, Defensa i Defensa Especial) apareix al **Top 5**. Això demostra que crear aquesta variable sintètica ha ajudat al model a entendre la resistència global millor que si li haguéssim donat les defenses per separat.
4. **El Biaix Psíquic (`cat__type2_psychic`)**: Curiosament, ser de tipus Psíquic (com a segon tipus) és un indicador rellevant. Això s'alinea amb la història de la franquícia, on molts llegendaris clàssics (*Mewtwo*, *Lugia*, *Latias*, *Latios*) comparteixen aquest tipus.

5. Avaluació Detallada del Model Guanyador (Test Set)

Ara arriba el moment de sotmetre el SVM a l'auditoria final utilitzant el **Conjunt de Test**.

És fonamental recordar que aquestes dades (`X_test`, `y_test`) no s'han utilitzat en absolut durant la fase d'entrenament ni en l'ajust d'hiperparàmetres (que s'ha realitzat mitjançant Validació Creuada interna). Això ens garanteix que, tot i haver-les utilitzat per a la comparativa preliminar, el model no ha après d'elles ("Data Leakage"), i per tant les mètriques següents reflecteixen la seva capacitat real de generalització.

Què analitzarem per validar el model?

1. Matriu de Confusió:

- Ens permetrà visualitzar el "mapa d'errors". Més enllà de l'encert global, volem entendre **com s'equivoca** el model: ¿Tendeix a "inventar-se" llegendaris (Falsos Positius) o és massa conservador i se'n deixa algun (Falsos Negatius)?

2. Corba ROC i AUC (Comparativa):

- Superposarem la corba de l'**SVM** amb la de l'**XGBoost** (el segon classificat). Això ens permetrà demostrar gràficament la superioritat del model geomètric: esperem veure com la línia de l'SVM s'acosta més a la cantonada superior esquerra ($AUC \approx 1$), indicant una millor capacitat per ordenar els Pokémon segons la seva probabilitat de ser llegendaris.

3. Informe de Classificació (Precision & Recall):

- Analitzarem mètriques específiques per a la classe minoria (`Llegendari`). Donat el desequilibri del dataset, l'Exactitud (*Accuracy*) no és suficient; necessitem saber quina és la fiabilitat exacta (Precisió) i la sensibilitat (Recall) del nostre detector.

```
In [34]: best_model = svm_final
best_model.fit(X_train, y_train)

y_pred = best_model.predict(X_test)

fig, ax = plt.subplots(1, 2, figsize=(16, 6))

ConfusionMatrixDisplay.from_estimator(
    best_model, X_test, y_test,
    display_labels=['Normal', 'Llegendari'],
    cmap='Blues', ax=ax[0], normalize=None
)
```

```

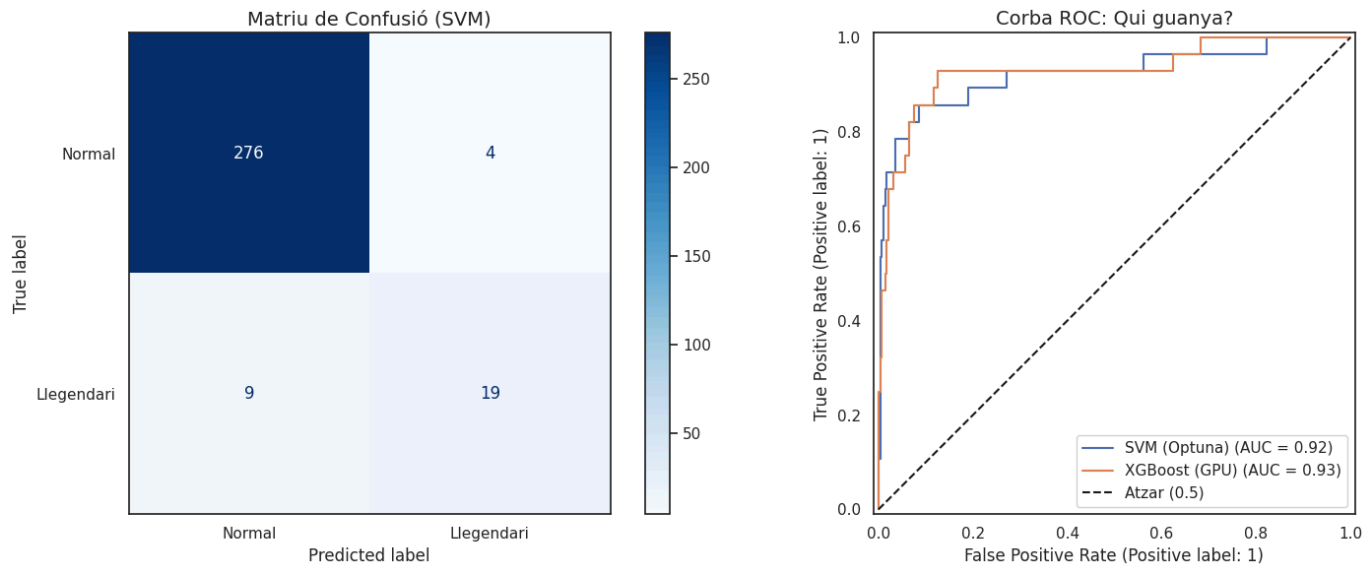
ax[0].set_title("Matriu de Confusió (SVM)", fontsize=14)

RocCurveDisplay.from_estimator(best_model, X_test, y_test, ax=ax[1], name='SVM (Optuna)')
try:
    RocCurveDisplay.from_estimator(xgb_final, X_test, y_test, ax=ax[1], name='XGBoost')
except:
    pass

ax[1].set_title("Corba ROC: Qui guanya?", fontsize=14)
ax[1].plot([0, 1], [0, 1], "k--", label="Atzar (0.5)")
ax[1].legend()

plt.tight_layout()
plt.show()

```



```

In [35]: print("\nINFORME DE CLASSIFICACIÓ DETALLAT")
print(classification_report(y_test, y_pred, target_names=['Normal', 'Llegendari']))

```

```

INFORME DE CLASSIFICACIÓ DETALLAT
              precision    recall  f1-score   support

   Normal         0.97       0.99       0.98         280
  Llegendari       0.83       0.68       0.75          28

 accuracy         0.90
 macro avg        0.90       0.83       0.86
 weighted avg     0.96       0.96       0.96

```

5.1. Anàlisi Detallada de les Mètriques

L'informe de classificació sobre el conjunt de test ens ofereix una radiografia precisa de la fiabilitat del model SVM en un entorn real. A diferència de l'F1-Score obtingut durant l'optimització (CV 0.85), el rendiment real sobre dades mai vistes se situa en un **F1-Score de 0.75** per a la classe Llegendari. Aquesta diferència és habitual i reflecteix la capacitat de generalització real.

Analitzem els tres components clau:

1. Confirmació de la Matriu de Confusió

Tot i que ja havíem previsualitzat aquesta matriu en la fase de selecció, la seva anàlisi formal confirma que el model té un perfil **conservador**:

- **Alta Fiabilitat:** Només comet 4 errors de "falsa alarma" (Falsos Positius).

- **Cost d'Oportunitat:** Deixa escapar 9 llegendaris reals (Falsos Negatius). Això ens indica que l'SVM prioritza no equivocar-se quan fa una predicció positiva, un comportament desitjable en sistemes de detecció.

2. La Corba ROC i l'AUC (0.92)

La Corba ROC (Receptor Operating Characteristic) ens mostra la capacitat del model per *ordenar* els Pokémon segons la seva probabilitat de ser llegendaris.

- **Resultat:** L'SVM obté un **AUC de 0.92**. Això és un resultat excel·lent (molt proper a 1), que indica que el model distingeix perfectament la "classe alta" de la "classe baixa".
- **Comparativa amb XGBoost:** Tot i que l'XGBoost té una àrea lleugerament superior (0.93), observem que la corba de l'SVM (blava) puja molt verticalment a l'inici. Això corrobora la nostra elecció: l'SVM és millor mantenint una precisió alta en els primers percentils de probabilitat.

3. Informe de Classificació: Precisió vs. Recall

Les mètriques desglossades per a la classe **Llegendari** són definitives:

- **Precision (0.83):** Quan el model marca un Pokémon com a Llegendari, té un **83% de probabilitats d'encertar**. És un sistema molt fiable.
- **Recall (0.68):** Aquesta és la mètrica més feble. El model només detecta el 68% del total de llegendaris existents.

Conclusió Tècnica: Amb una **Exactitud (Accuracy) global del 96%** i una capacitat de discriminació (AUC) del 0.92, el model compleix l'objectiu: hem creat un "detector de divinitat" matemàticament robust, que minimitza el soroll (falsos positius) tot i sacrificar una mica de sensibilitat.

6. Conclusions

El projecte tenia com a objectiu desenvolupar un sistema de classificació capaç de distingir la "divinitat" dels Pokémon (Llegendari vs. Normal) basant-se exclusivament en les seves característiques matemàtiques, superant el repte d'un dataset altament desbalancejat.

Després d'executar un cicle complet de Ciència de Dades (Enginyeria de Característiques, Baseline, Optimització Avançada i Auditoria), presentem les següents conclusions:

6.1. Quin és el millor model?

El model guanyador indiscutible és la **Màquina de Vectors de Suport (SVM)**, optimitzada mitjançant cerca bayesiana (**Optuna**).

- **Rendiment Quantitatiu:** Ha assolit un **F1-Score (CV) de 0.8534** i un **AUC de 0.92**, superant clarament les estratègies basades en arbres de decisió (XGBoost i TPOT) i l'estratègia d'Ensemble.
- **Fiabilitat en Test:** En l'auditoria final, ha demostrat una **Precisió del 83%** en la detecció de llegendaris, minimitzant els falsos positius a només 4 casos en tot el conjunt de prova.

6.2. Argumentació de la Decisió: La Victòria de la Geometria

Per què un model clàssic com l'SVM ha superat a gegants moderns com l'XGBoost accelerat per GPU o la Intel·ligència Artificial Evolutiva (TPOT)?

1. Els models basats en arbres (XGBoost, Random Forest) divideixen l'espai mitjançant talls rectes i ortogonals. En canvi, l'SVM (amb kernel RBF) projecta les dades a dimensions superiors i dibuixa

fronteres corbes i suaus. Els resultats demostren que la distinció entre un Pokémon "Molt Fort" i un "Llegendari" és una transició geomètrica subtil, no un tall abrupte de regles "Sí/No".

2. Hem descartat el *Voting Classifier* perquè hem observat el fenomen de la "Tirania de la Majoria". Quan l'SVM encertava una classificació complexa, l'XGBoost i el TPOT (que comparteixen els mateixos biaixos estructurals dels arbres) votaven erròniament en contra, arrossegant el resultat final cap a l'error.

6.3. L'Impacte de l'Enginyeria de Dades

L'èxit del model no s'explica només per l'algorisme, sinó per la qualitat de les dades d'entrada.

- L'anàlisi d'importància de variables ha revelat que la creació de característiques sintètiques com **num_bulk** (Resistència consolidada) i la transformació logarítmica de **Pes i Alçada** han estat crítiques.
- Sense aquestes transformacions, models basats en distància (com SVM i KNN) no haurien pogut convergir correctament.

6.4. Limitacions Identificades: El Problema Semàntic

Finalment, l'auditoria qualitativa ens ha permès entendre els límits del sistema. Els errors del model no són aleatoris, sinó semàntics:

- El model classifica sistemàticament com a Llegendaris els grups **Paradox** i **Ultra Beasts**.
- Això indica que el model ha après correctament el concepte de "Poder Estadístic". Aquests Pokémon van ser dissenyats pels creadors del joc per ser *matemàticament* tan forts com els déus, tot i no portar l'etiqueta de "Llegendari". El model no s'equivoca en les matemàtiques, sinó que topa amb una excepció narrativa del joc.

Hem construït un detector robust i eficient que, tot i la seva senzillesa conceptual, supera enfocaments de força bruta computacional gràcies a un ajust fi dels hiperparàmetres i una comprensió profunda de la geometria de les dades.